Solana Blinks: Technical White Paper

Version: 1.0
Date: April 20, 2026
Project: Blink Anti-Gravity (Reference Implementation)
Status: Production-Ready

Executive Summary

> **For Business:** Blinks remove the friction from blockchain transactions on social media. Today, converting a social media post into a funded blockchain action requires users to copy wallet addresses, switch applications, paste information, and verify transactions across multiple interfaces. Blinks enable direct transaction execution without leaving the user's social feed, dramatically improving adoption and engagement.

Solana Blinks are trustless, shareable blockchain actions embedded directly into social media interfaces and compatible clients. This white paper documents the architecture, implementation, security model, and use cases for the Blink standard, using the "Donate SOL" reference implementation as a practical case study.

Key Innovations: - **Frictionless**: Execute Solana transactions without wallet-switching or manual verification - **Discoverable**: Works natively on X.com and across blockchain wallets with metadata-driven unfurling - **Secure**: Cryptographically verified transactions with optional transaction previews - **Standards-Based**: Built on Dialect's `@solana/actions` specification for interoperability - **Portable**: Operate across platforms via direct URLs or embedded Blink clients

Document Audience: Both technical practitioners implementing Blinks and business stakeholders evaluating adoption.

---

1. Problem Statement

1.1 The Blockchain Transaction Friction Problem

The blockchain ecosystem boasts billions in daily transaction volume, yet adoption remains constrained in mainstream applications. A critical pain point: **the conversion gap between social discovery and transaction execution**.

Current Workflow (Pre-Blinks)

```
User sees post about fundraiser → Copies wallet address → Switches
→ Pastes address → Enters amount → Reviews transaction → Approves →
→ Returns to original app
```

Friction Points: 1. **Context Switching**: Minimum 2-3 app switches (browser → wallet → confirmation → browser) 2. **Manual Verification**: Users manually verify copied addresses (phishing risk) 3. **Copy-Paste Errors**: Typos in addresses lose funds permanently 4. **Knowledge Barrier**: Required understanding of wallet structure, addresses, and transaction mechanics 5. **Time Commitment**: 2-5 minutes per transaction vs. milliseconds for traditional payments 6. **Discovery Friction**: Blockchain actions invisible until user manually shares payment details

Real-World Impact

- **Creator Monetization**: Streamers can't monetize directly during broadcast
- **Charitable Giving**: Donation friction suppresses participation (mobile users especially impacted)
- **Governance**: Token voting requires off-platform transaction construction
- **Commerce**: Markets cannot embed checkout flows in social platforms
- **Event Ticketing**: Ticket distribution requires manual blockchain interactions

1.2 Why Traditional Solutions Don't Solve This

Solution	Limitation
Custodial Wallets	Centralized risk, asset seizure potential, regulatory uncertainty
QR Codes	Requires wallet app install, still requires manual amount entry
Email/SMS Payment Links	Custody model introduces counterparty risk
API Integrations	Requires application to hold user keys (high security risk)
Mobile Apps	Ecosystem fragmentation, discoverability limited

1.3 The Adoption Equation

Blockchain platforms achieve mainstream adoption when:

$$\text{Adoption Rate} \propto \frac{\text{Transaction Utility}}{\text{User Friction}}$$

Current State: High utility but **extremely high friction** prevents mainstream adoption.

Blinks Goal: Maintain utility while reducing friction to near-zero.

---

2. Solution: Blinks Explained

2.1 What Are Blinks?

Blinks are **standardized, shareable, on-chain actions that can be triggered with a single click from social media feeds**. They are:

- **Native to protocol**: Operate through standardized endpoints and metadata

- **Client-agnostic**: Work in X.com, Phantom, Backpack, and any Blink-compatible application
- **Non-custodial**: User controls private keys throughout; no intermediary holds funds
- **Composable**: Blink endpoints can chain transactions or propose complex interactions
- **Discoverable**: Metadata-driven with rich UI support for previews and confirmations

2.2 The Blink Architecture (Simplified)

```
┌──────────────────┐          ┌──────────────────┐
│   Social Media   │          │  Blockchain      │
│   (X.com)        │          │  (Solana Devnet) │
└──────────┬───────┘          └──────────┬───────┘
           │                             ▲
           │ User shares Blink URL       │
           │                             │
           ▼                             │
      ┌──────────────────────────┐       │
      │ Blink Client             │       │
      │ - Fetches action metadata │       │
      │ - Renders UI (buttons)   │       │
      │ - Constructs transaction │──────┘
      └──────────┬───────────────┘
                 │
                 │ Requests action endpoint
                 │
                 ▼
      ┌──────────────────────────┐
      │ Blink Provider           │
      │ (Your API)               │
      │ - GET: Metadata          │
      │ - POST: Build transaction │
      └──────────────────────────┘
```

2.3 How Blinks Solve the Problem

Problem	Blink Solution	Outcome
Context switching	Embedded in social feed	0 app switches
Manual verification	Cryptographic signatures	No phishing
Copy-paste errors	Automatic address handling	100% accuracy
Knowledge barrier	One-click UI	Non-technical accessible
Time commitment	Single click submission	<1 second
Action discovery	Metadata-driven unfurling	Passive discovery

2.4 Integration Approaches

Approach 1: Native X.com Support ✨

When X.com supports Blinks natively:

```
User shares URL → X crawls OG meta tags → X recognizes Solana Actio
→ X injects interactive Blink UI directly in feed → User clicks butt
→ Transaction executes without leaving feed
```

Status: X.com recognizes action URLs; full UI support in development.

Approach 2: Chrome Extension (Current Production)

For broader client support:

```
User shares URL → Chrome extension detects link → Extension fetches
→ Extension injects Blink UI component → User clicks button
→ Transaction executes with wallet confirmation
```

Status: ✅ Active and tested in production.

Approach 3: Embedded Blink Client

Direct integration:

```
URL with ?action= parameter → Blink client page loads → Client fetc
→ Renders full Blink UI → User interacts → Transaction executes
```

Status: ✅ Implemented in this reference project at `/blink` endpoint.

---

3. Architecture & Technical Design

3.1 System Components

Component 1: Blink Provider

- Your API/application that exposes Solana action endpoints
- Responsible for: transaction construction, metadata generation, validation
- Lives at: `/api/actions/[action-name]`
- Returns: Standardized JSON following `@solana/actions` specification

Component 2: Blink Client

- Consumer application (X.com, Phantom, Backpack, or dedicated page)
- Fetches metadata from Blink Provider
- Renders interactive UI (buttons, inputs, previews)
- Constructs and submits transactions to user's wallet

Component 3: Dialect SDK (`@solana/actions`)

- Standardizes action specification and routing
- Provides TypeScript types for request/response formats
- Handles CORS headers and browser preflight
- Reference: Dialect Labs maintains this standard

Component 4: User's Wallet

- Signs transactions (Phantom, Backpack, Ledger, etc.)
- Controls private keys
- Submits signed transaction to Solana network

3.2 Data Flow Diagram

```
 DISCOVERY PHASE:
──────────────────
1. User sees post with Blink URL
2. Blink Client (extension or app) detects URL
3. Client sends: OPTIONS /api/actions/donate-sol (CORS preflight)
4. Provider responds with CORS headers + blockchain info

METADATA PHASE:
────────────────
5. Client sends: GET /api/actions/donate-sol
6. Provider responds: {
     title: "Donate SOL"
     icon: "https://..."
     actions: [
       { label: "Donate 0.1 SOL", postUrl: "..." },
       { label: "Custom amount", type: "input" }
     ]
   }
7. Client renders UI with buttons

USER INTERACTION PHASE:
────────────────────────
8. User clicks button or enters custom amount
9. Client sends: POST /api/actions/donate-sol?amount=0.50
10. Provider constructs:
    - Creates transfer instruction
    - Builds versioned transaction
    - Encodes in base64
    - Returns to client

TRANSACTION SIGNING PHASE:
───────────────────────────
11. Client passes transaction to user's wallet
12. Wallet displays preview + approval dialog
13. User reviews and signs
14. Wallet broadcasts to Solana network
```

```
CONFIRMATION PHASE:
────────────────────
14. Transaction lands on-chain with user's signature
15. Client displays success confirmation
16. Provider logs transaction (optional)
```

3.3 Key Design Patterns

Pattern 1: Action Discovery (GET)

Clients fetch metadata to determine available actions and UI elements.

```
 GET https://blink-provider.com/api/actions/donate-sol

Response:
{
  type: "action",
  icon: "https://...",
  title: "Donate SOL",
  description: "Support our mission",
  links: {
    actions: [
      { label: "Donate 0.1 SOL", href: "..." },
      { label: "Donate 0.5 SOL", href: "..." },
      {
        label: "Custom",
        href: "...",
        parameters: [
          { name: "amount", label: "Amount in SOL" }
        ]
      }
    ]
  }
}
```

Pattern 2: CORS Preflight (OPTIONS)

Browsers require preflight for cross-origin requests.

```
 OPTIONS /api/actions/donate-sol

Response Headers:
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, OPTIONS, POST
x-blockchain-ids: solana:5eykt4UsFv2P6ysS48Dxi6scope49oFykjQxf8Jm6NE
```

Pattern 3: Transaction Construction (POST)

Client requests signed transaction.

```
 POST /api/actions/donate-sol
Content-Type: application/json
{
  "account": "user_wallet_address",
  "amount": "0.50"
}

Response:
{
  transaction: "base64_encoded_versioned_transaction",
  message: "Transaction ready for signing",
  error?: "Human-readable error if failed"
}
```

3.4 Endpoint Relationships

```
discovery endpoint
│
└─ /actions.json
   Describes site structure
   Maps paths to action endpoints

   Example: { rules: [{ pathname: "/donate", apiPath: "/api/actions/

action endpoints (these are interconnected)
│
├─ OPTIONS /api/actions/donate-sol
│  Purpose: CORS preflight to enable browser access
│
├─ GET /api/actions/donate-sol
│  Purpose: Fetch metadata (title, icon, button definitions)
│  Builds UI without requiring wallet connection
│
└─ POST /api/actions/donate-sol
   Purpose: Construct transaction for signing
   Requires: account (public key), query params (amount, etc.)
   Returns: Base64-encoded transaction ready for wallet
```

---

4. API Endpoints & Specifications

> **For Developers:** This section details the exact request/response formats for implementing a Blink Provider.

4.1 Discovery: `/actions.json`

Purpose: Define how website URLs map to Solana action endpoints.

Request:

```
GET https://your-blink-provider.com/actions.json
```

Response:

```
{
  "rules": [
    {
      "pathname": "/donate-sol",
      "apiPath": "/api/actions/donate-sol"
    },
    {
      "pathname": "/",
      "apiPath": "/api/actions/donate-sol"
    }
  ]
}
```

When Used: Clients may use this to discover available actions on a domain.

4.2 CORS Preflight: `OPTIONS /api/actions/[action]`

Purpose: Enable browser-based clients to make cross-origin requests.

Request:

```
OPTIONS /api/actions/donate-sol HTTP/1.1
Host: your-blink-provider.com
Origin: https://x.com
Access-Control-Request-Method: POST
```

Response Headers:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, OPTIONS, POST
Access-Control-Allow-Headers: Content-Type
Access-Control-Max-Age: 86400
x-blockchain-ids: solana:5eykt4UsFv2P6ysS48Dxi6scope49oFykjQxf8Jm6NE
```

Response Body:

```
{
  "type": "action",
  "icon": "https://your-domain.com/icon.png",
  "title": "Donate SOL",
  "description": "Support our mission",
  "links": {
    "actions": []
  }
}
```

4.3 Metadata & UI Definition: `GET /api/actions/[action]`

Purpose: Provide title, icon, description, and available interaction buttons.

Request:

```
GET /api/actions/donate-sol HTTP/1.1
Host: your-blink-provider.com
Accept: application/json
```

Response:

```
{
  "type": "action",
  "icon": "https://your-domain.com/donate-sol.jpg",
  "title": "Donate SOL to Our Mission",
  "description": "Every SOL helps us build. Choose your amount or er
  "label": "Donate",
  "links": {
    "actions": [
      {
        "label": "Donate 0.1 SOL",
        "href": "/api/actions/donate-sol?amount=0.1"
      },
      {
        "label": "Donate 0.5 SOL",
        "href": "/api/actions/donate-sol?amount=0.5"
      },
      {
        "label": "Donate 1.0 SOL",
        "href": "/api/actions/donate-sol?amount=1.0"
      },
      {
        "label": "Custom Amount",
        "href": "/api/actions/donate-sol",
        "parameters": [
          {
            "name": "amount",
            "label": "Amount to donate (SOL)",
            "required": true,
            "type": "number",
            "min": "0.01",
            "max": "1000",
            "step": "0.01"
          }
        ]
      }
    ]
```

```
  }
}
```

Field Definitions: - `type`: Always `"action"` for Blink endpoints - `icon`: URL to image (JPG, PNG) - `title`: Displayed heading (max 30 chars recommended) - `description`: Context for user (max 100 chars recommended) - `label`: Button text if embedded ("Donate", "Vote", etc.) - `links.actions`: Array of buttons/inputs user can interact with - `href`: URL to POST to (includes query params for fixed amounts) - `parameters`: Input fields for custom values

---

4.4 Transaction Construction: `POST /api/actions/[action]`

Purpose: Build a transaction ready for user's wallet to sign.

Request:

```
 POST /api/actions/donate-sol?amount=0.5 HTTP/1.1
Host: your-blink-provider.com
Content-Type: application/json

{
  "account": "7eYdLSn7zVrHjGWBo8xPYfCvLDTxqRFfKvqLYk8RG8Vz",
  "amount": "0.5"
}
```

Request Parameters: - `account` (body): Public key of user initiating action (base58) - `amount` (query): Amount in SOL (from button or user input)

Response (Success):

```
 {
  "transaction": "AgADCr4I/gTpb6bPGqeFRnfxZxFvHKuGcYSrUTFLDpN...",
  "message": "Review your donation of 0.5 SOL"
}
```

Response (With Validation):

```
{
  "transaction": "AgADCr4I/gTpb6bPGqeFRnfxZxFvHKuGcYSrUTFLDpN...",
  "message": "Review your donation of 0.5 SOL",
  "links": {
    "next": {
      "href": "/api/actions/donate-sol/confirm?tx=AgADCr4I...",
      "label": "Confirm donation"
    }
  }
}
```

Response (Error):

```
{
  "error": "Amount exceeds maximum donation limit (1000 SOL)"
}
```

Transaction Format: - Transactions are base64-encoded Solana versioned transactions - Must be signable by user's wallet immediately - Should use `getLatestBlockhash()` to ensure freshness - Should NOT be signed by provider (non-custodial principle)

4.5 Error Handling Convention

Consistent error responses across all endpoints:

```
{
  "error": "Human-readable error description",
  "details": "Technical details or error code",
  "links": {
    "next": {
      "href": "/api/actions/donate-sol",
      "label": "Try again or view help"
    }
  }
}
```

Common Errors: - `400` : Invalid parameters (amount out of range, invalid account) - `429` : Rate limit exceeded - `500` : Server error (RPC connection failed, transaction construction failure) - `503` : Temporary unavailable (chain congestion)

---

5. Reference Implementation: The Donate SOL Blink

This project implements a complete Blink for accepting Solana donations. Here's how all pieces fit together.

5.1 Project Structure

```
src/
├── app/
│   ├── page.tsx                          # Landing page
│   ├── donate-sol/
│   │   ├── page.tsx                      # HTML page with OG tags
│   │   └── client.tsx                    # React component for UI
│   ├── blink/
│   │   ├── page.tsx                      # Blink client page
│   │   └── client.tsx                    # Blink UI renderer
│   ├── api/
│   │   ├── donate/
│   │   │   └── route.ts                  # Direct action endpoint
│   │   └── actions/donate-sol/
│   │       └── route.ts                  # Spec-compliant endpoint
│   └── actions.json/
│       └── route.ts                      # Discovery endpoint
└── ...
```

5.2 Three Ways to Share Your Blink

Option A: Direct Action URL (Recommended for X.com)

```
https://blink-with-anti.vercel.app/api/donate
```

- Returns action JSON directly
- Extension detects immediately
- No page navigation needed
- Instant unfurling on X.com

Option B: Blink Client URL (Web-friendly)

```
https://blink-with-anti.vercel.app/blink?action=solana-action%3Ahtt
```

- Full Blink UI in dedicated page
- Works in non-extension clients
- Can be opened in any browser

Option C: HTML Page URL (Most discoverable)

```
https://blink-with-anti.vercel.app/donate-sol
```

- SEO-friendly, shows preview card
- Has interactive UI when visited directly
- Extension can detect via OG meta tags

5.3 Endpoint Implementation Details

Endpoint 1: `/app/api/actions/donate-sol/route.ts`

Spec-compliant Solana Actions endpoint:

```
 import { ActionGetResponse, ActionPostRequest, ACTIONS_CORS_HEADERS
import { Connection, PublicKey, SystemProgram, Transaction, Versione

const DONATION_WALLET = "4GN5mEZfbMUHWEWCzUEJ..."; // Destination wa
const RPC_URL = "https://api.devnet.solana.com";

export async function OPTIONS(request: Request) {
  return new Response(null, { headers: ACTIONS_CORS_HEADERS });
}

export async function GET(request: Request): Promise<Response> {
  const baseUrl = new URL(request.url).origin;

  const payload: ActionGetResponse = {
    type: "action",
    icon: `${baseUrl}/donate-sol.jpg`,
    title: "Donate SOL",
    description: "Support our mission with a Solana donation",
    label: "Donate",
    links: {
      actions: [
        { label: "Donate 0.1 SOL", href: "/api/actions/donate-sol?am
        { label: "Donate 0.5 SOL", href: "/api/actions/donate-sol?am
        {
          label: "Custom Amount",
          href: "/api/actions/donate-sol",
          parameters: [
            { name: "amount", label: "Amount (SOL)", required: true
          ]
        }
      ]
    }
  };

  return new Response(JSON.stringify(payload), {
    headers: ACTIONS_CORS_HEADERS,
    status: 200
```

```
  });
}

export async function POST(request: Request): Promise<Response> {
  try {
    const { searchParams } = new URL(request.url);
    const amount = parseFloat(searchParams.get("amount") || "0");
    const { account } = await request.json();

    if (!account) throw new Error("Missing account");
    if (amount < 0.01 || amount > 1000) throw new Error("Invalid amo

    const connection = new Connection(RPC_URL);
    const userPubkey = new PublicKey(account);
    const destinationPubkey = new PublicKey(DONATION_WALLET);

    // Create transfer instruction
    const instruction = SystemProgram.transfer({
      fromPubkey: userPubkey,
      toPubkey: destinationPubkey,
      lamports: amount * 1_000_000_000  // Convert SOL to lamports
    });

    // Get latest blockhash
    const { blockhash } = await connection.getLatestBlockhash();

    // Create and sign transaction
    const transaction = new Transaction({
      recentBlockhash: blockhash,
      feePayer: userPubkey
    }).add(instruction);

    // Serialize and encode
    const serialized = transaction.serialize({ requireAllSignatures:
    const base64 = serialized.toString("base64");

    return new Response(
      JSON.stringify({
```

```
        transaction: base64,
        message: `Thank you! Donating ${amount} SOL...`
      }),
      { headers: ACTIONS_CORS_HEADERS }
    );
  } catch (error) {
    return new Response(
      JSON.stringify({ error: (error as Error).message }),
      { headers: ACTIONS_CORS_HEADERS, status: 400 }
    );
  }
}
```

Endpoint 2: `/app/api/donate/route.ts`

Direct action endpoint (simplified):

```
export async function GET(request: Request) {
  const baseUrl = new URL(request.url).origin;

  return new Response(
    JSON.stringify({
      type: "action",
      icon: `${baseUrl}/donate-sol.jpg`,
      title: "Donate SOL",
      // ... same as above
    }),
    { headers: { "Content-Type": "application/json" } }
  );
}
```

Endpoint 3: `/app/donate-sol/page.tsx`

HTML page with OG meta tags:

```
import Head from "next/head";

export default function DonateSolPage() {
  return (
    <>
      <Head>
        <meta property="og:title" content="Donate SOL — Solana Blink
        <meta property="og:description" content="Support our mission
        <meta property="og:image" content="https://blink-with-anti.v
        <meta name="solana:action:apiUrl" content="https://blink-wit
      </Head>
      <div className="donate-container">
        {/* Interactive UI renders here */}
      </div>
    </>
  );
}
```

5.4 How Requests Flow Through the System

```
 Chrome Extension detects: https://blink-with-anti.vercel.app/api/do
        ↓
OPTIONS /api/donate (CORS preflight) → 200 OK + ACTIONS_CORS_HEADERS
        ↓
GET /api/donate → Returns metadata JSON + button definitions
        ↓
Extension renders UI with buttons
        ↓
User clicks "Donate 0.5 SOL"
        ↓
POST /api/donate?amount=0.5 with { account: "user_key" }
        ↓
Server constructs transaction:
  - Creates SystemProgram.transfer instruction
  - Gets latest blockhash
  - Builds versioned transaction
  - Serializes to base64
        ↓
Response: { transaction: "base64...", message: "Thank you!" }
        ↓
Extension passes to Phantom wallet
        ↓
User signs in Phantom
        ↓
Wallet broadcasts to Solana devnet
        ↓
Transaction lands on-chain
        ↓
"Thank you!" confirmation displays
```

---

6. Security Considerations

> **For Business:** Blinks use the same cryptography as all Solana transactions. Users sign, not trust.

> **For Developers:** This section covers threat model, mitigation strategies, and production-ready patterns.

6.1 Threat Model

Threat 1: Malicious Action URLs

Risk: Attacker posts a URL that claims to be a donation but steals funds.

Mitigation: - Users must verify destination wallet address in wallet preview - Wallet displays full transaction before signing - Cryptographic signature prevents transaction modification - Never display signed transactions without wallet confirmation

Threat 2: Phishing via Social Engineering

Risk: Convincing clone blink directing to attacker wallet.

Mitigation: - Blinks are URL-based; cryptography cannot be faked - User education: "Verify wallet address matches official sources" - Brand verification in Blink metadata (could link to social proof) - Extension/wallet can implement domain whitelist (optional)

Threat 3: Transaction Replay

Risk: Intercepted signed transaction replayed on different account/chain.

Mitigation: - Solana uses `blockhash` (expires after ~2 minutes) - Destination address hardcoded in transaction - Each transaction includes unique fee payer - Replay across chains impossible due to `chainId` in transaction header

Threat 4: Provider Goes Offline/Censors Transactions

Risk: Provider refuses to construct transactions for certain users.

Mitigation: - Transaction construction is deterministic; users can verify provider's math - Alternative providers can serve same action - No permission/allowlist required; any provider can host action endpoint

Threat 5: RPC Endpoint Manipulation

Risk: Malicious RPC returns fake blockhash or transaction state.

Mitigation: - Use trusted RPC endpoints (Solana Foundation, Alchemy, QuickNode) - User's wallet verifies transaction against their chosen network state - Impossible to fake valid signatures without private key

6.2 Safe Action Discovery Patterns

```
// SAFE: Verified metadata sources
1. Official domain (brand DNS)
2. Verified social accounts with linked action URLs
3. Community-curated registries with attestations
4. On-chain program metadata (for program-based actions)

// CAUTION: Requires user verification
1. Email links (verify sender is authentic)
2. Unknown social posts (user should verify wallet address)
3. QR codes (user should verify domain)

// UNSAFE: Avoid these patterns
1. Shortened URLs without verification (bit.ly, etc.)
2. Microsites claiming to offer actions
3. Unverified third-party Blink registries
```

6.3 Provider Security Checklist

- ✅ **HTTPS only** — All endpoints use Transport Layer Security
- ✅ **No private key handling** — Never touch user's private keys
- ✅ **Validate inputs** — Amount ranges, account format, query parameters
- ✅ **Rate limiting** — Prevent abuse (POST /api/actions/* limited to 10 req/sec per IP)
- ✅ **Fresh blockhash** — Call `getLatestBlockhash()` at transaction time (not cached)
- ✅ **Correct destination** — Hardcode destination wallet, never trust user input
- ✅ **Monitoring** — Log: timestamps, account addresses (hashed), amounts, error rates
- ✅ **CORS headers properly configured** — Only expose necessary methods

- ✅ **Error messages safe** — Don't leak internal details, use generic errors

6.4 Wallet Security Checklist

> **For Users:** - ✅ Always review transaction details before signing - ✅ Verify destination wallet matches expected address (check first 4 and last 4 chars) - ✅ Check amount is what you intended - ✅ Familiar with wallet's UI for confirmation dialogs - ✅ Never authorize unlimited approvals (always check scope) - ✅ Use hardware wallet for large transactions

---

7. Deployment & Testing

7.1 Prerequisites

Local Development: - Node.js 18+ - npm or bun - Solana CLI (for validation)

Infrastructure: - Solana RPC endpoint (Devnet: https://api.devnet.solana.com) - Solana wallet with devnet SOL (for testing) - HTTPS domain + SSL certificate - Vercel account (or alternative Node.js hosting)

7.2 Environment Configuration

Create `.env.local`:

```
 # RPC Endpoints
NEXT_PUBLIC_SOL_RPC_URL=https://api.devnet.solana.com

# Donation wallet (your destination)
NEXT_PUBLIC_DONATION_WALLET=5j9...abc123

# For production mainnet (later):
# NEXT_PUBLIC_SOL_RPC_URL=https://api.mainnet-beta.solana.com
```

7.3 Build & Deploy Steps

Step 1: Build Next.js App

```
npm install
npm run build

# Check output
# ✓ Build complete
# ✓ Routes analyzed
# ✓ API routes: /api/actions/donate-sol, /api/donate, /actions.json
```

Step 2: Test Locally

```
npm run dev
# Visit http://localhost:3000

# Test endpoints:
curl http://localhost:3000/api/actions/donate-sol
curl -X OPTIONS http://localhost:3000/api/actions/donate-sol
curl -X POST http://localhost:3000/api/actions/donate-sol \
  -H "Content-Type: application/json" \
  -d '{"account":"7eYdLSn7zVrHjGWBo8xPYfCvLDTxqRFfKvqLYk8RG8Vz"}'
```

Step 3: Deploy to Vercel

```
npm i -g vercel
vercel deploy --prod

# You'll get: https://your-project.vercel.app
```

Step 4: Verify Production

```
# Verify endpoint accessibility
curl https://your-project.vercel.app/api/actions/donate-sol

# Check CORS headers present
curl -I -X OPTIONS https://your-project.vercel.app/api/actions/donat
```

7.4 Chrome Extension Testing

```
cd chrome-extension

# Build extension
npm install
npm run build

# Load in Chrome:
# 1. Open chrome://extensions/
# 2. Enable "Developer mode" (top-right toggle)
# 3. Click "Load unpacked"
# 4. Select ./dist folder
# 5. Extension loaded!
```

Test on X.com: 1. Go to x.com 2. Navigate to your project's blink URL 3. Copy the URL from the browser 4. Post on X.com 5. Extension should detect and unfurl

7.5 Performance Considerations

Operation	Target	Actual	Notes
GET /api/actions/donate-sol	<100ms	~50ms	Metadata fetch
POST /api/actions/donate-sol	<500ms	~200-300ms	RPC blockhash fetch
Transaction signing	<1s	User-dependent	Wallet confirmation
Transaction finalization	<10s	~5s devnet	Network confirms

Optimization strategies: - Cache OG image URLs (CDN) - Connection pooling to RPC (use `@solana/web3.js` built-in) - Implement request deduplication (same amount requests in flight) - Monitor RPC response times, switch on timeout

7.6 Production Checklist

- [ ] **Endpoints tested with curl**
- [ ] **CORS headers verified** with OPTIONS
- [ ] **HTTPS enforced** (no http:// in production)
- [ ] **RPC endpoint stable** (test failover strategy)
- [ ] **Destination wallet confirmed** (double-check, immutable in code)
- [ ] **Rate limiting configured** (prevent brute force)
- [ ] **Error handling graceful** (no exception stack traces to user)
- [ ] **Logging enabled** (for debugging)
- [ ] **Chrome extension deployed** or X.com native support verified
- [ ] **Monitoring alerts set** (endpoint health, RPC latency)

---

8. Real-World Use Cases & Examples

8.1 Charitable Giving

Current Friction: Donors visit website → create account → fill donation form → process payment → wait for confirmation.

Blink Solution:

```
 Charity posts link on X.com
   ↓
Follower clicks "Donate 10 SOL"
   ↓
Phantom wallet opens automatically
   ↓
Follower confirms signature
   ↓
Donation lands on-chain
   ↓
Charity displays thank-you message with TxHash

Time: <30 seconds (vs. 5-10 minutes without Blink)
```

Production Implementation: - `/api/actions/donate` returns fixed amounts (0.1, 1, 5, 10 SOL) - Optional custom amount input - Metadata includes charity's mission and fund allocation - POST endpoint validates (donation > 0.01 SOL, < 100 SOL max) - Transaction success links to Solscan explorer

8.2 Creator Monetization

Current Friction: Streamers cannot monetize directly during broadcasts; viewers must leave stream.

Blink Solution:

```
 Streamer includes Blink URL in chat
   ↓
Chat message says "Support me: [blink-url]"
   ↓
Viewer clicks "Send $5 worth of SOL"
   ↓
SOL lands in streamer's wallet
   ↓
Streamer sees real-time notification in OBS overlay

Outcome: Direct monetization during stream, no checkout friction
```

Implementation Variation: - Dynamic Blink URL updates tip amount based on streamer's settings - Backend validates streamer address (prevents scams) - Returns transaction for specific streamer's wallet - Webhook notification when tip lands

8.3 Governance & Voting

Current Friction: Token voting requires visiting governance portal, verifying proposals, confirming votes separately.

Blink Solution:

```
 DAO posts voting proposal on X.com with Blink URLs
   ↓
"Vote YES" → Blinks to /api/actions/vote-yes?proposalId=42
"Vote NO" → Blinks to /api/actions/vote-no?proposalId=42
   ↓
Token holder clicks preference
   ↓
Voting instruction created and signed
   ↓
Vote registered on-chain in transaction
   ↓
Voter receives confirmation with voting power recorded

Outcome: Governance accessible directly from social media
```

Technical Notes: - `/api/actions/vote-yes` and `/api/actions/vote-no` are separate endpoints - Each constructs appropriate voting program instruction - Validates user owns governance tokens (via RPC query) - Returns error if user has already voted or tokens insufficient

8.4 Event Ticketing

Current Friction: Attendees fill form → pay via credit card → receive email ticket → verify at venue.

Blink Solution:

```
 Event organizer posts Blink URL
   ↓
Attendee clicks "Get Ticket - 3 SOL"
   ↓
Payment constructs:
   - Transfer 3 SOL to organizer
   - Mint NFT ticket to attendee's wallet
   ↓
Transaction includes both instructions
   ↓
Attendee now holds NFT (proof of ticket)
   ↓
At venue: Organizer scans attendee's NFT (verifies authenticity)

Outcome: Frictionless, verifiable tickets on-chain
```

Implementation Complexity: Medium - Requires two instructions (payment + NFT mint) - Transaction includes metadata program to store event details - Validates NFT collection is official (prevents counterfeits) - Optional: allowlist attendees, set per-wallet purchase limits

8.5 Trading/DeFi Swap

Current Friction: Navigate DEX app → connect wallet → select tokens → set slippage → confirm.

Blink Solution:

```
 Social post suggests trader opportunity
   ↓
"Swap 100 USDC → SOL" Blink
   ↓
Blink constructs optimized swap instruction
   ↓
Shows FX rate and slippage in preview
   ↓
Trader confirms
   ↓
Transaction executes atomically
   ↓
Trader now holds SOL (no counterparty risk)
```

Implementation Complexity: High - Requires composing DEX program instructions (Jupiter, Marinade, etc.) - Dynamic price quotes (may require API call to DEX) - Slippage tolerance (user customizable) - Transaction may fail if price moves (requires retry logic)

8.6 Other Emerging Use Cases

Use Case	Transaction Type	Status
Contest Entries	SOL + metadata	Live
Staking Pools	Delegate stake	In dev
NFT Minting	SPL token transfer	Live
Escrow Services	Multi-sig lock	In dev
Subscription Payments	Recurring (via program)	Planned
Lending/Borrowing	Collateral deposit	In dev

---

9. Roadmap & Future Enhancements

9.1 Short-Term (Next 3-6 Months)

Q2 2026: Mobile Wallet Integration

- Phantom Mobile native Blink support
- Backpack Mobile native Blink support
- Solflare deep linking
- **Impact**: 3-4x user base expansion (mobile-first market)

Q2 2026: Enhanced Metadata

- Transaction preview with human-readable instructions
- Estimated fee display pre-signing
- Transaction simulation results (show what happens without signing)
- **Impact**: User confidence increases, reduces failed transactions

Q3 2026: Blink Template Library

- Pre-built templates for common actions (donate, vote, mint, swap)
- Drag-and-drop builder for non-developers
- One-click deployment to GitHub Pages / Vercel
- **Impact**: Accessibility for non-technical creators

9.2 Medium-Term (6-12 Months)

Q3 2026: Multi-Chain Blinks

- Blinks that work on Polygon, Arbitrum, Base, Ethereum
- Unified Blink standard (not Solana-only)
- Cross-chain bridge actions
- **Impact**: Expand TAM from Solana users to all blockchain users

Q4 2026: Blink Marketplace/Registry

- Verified action directory (like GitHub Actions marketplace)
- Community-curated, with attestations
- Search for actions by category (charity, gaming, DeFi, etc.)
- **Impact**: Discoverability of trustworthy Blinks

Q4 2026: Advanced Security

- Blink signature verification (cryptographically attest to action authenticity)
- Risk scoring (rate actions by historical safety)
- Insurance/bonding for high-value actions
- **Impact**: Built-in trust without user research

9.3 Long-Term Vision (12+ Months)

2027: Ubiquitous Blockchain in Social Media

- X.com, Farcaster, Mirror, Bluesky native Blink support
- TikTok, Instagram Blink integration (ecosystem dependent)
- Email clients support Blink URLs
- **Impact**: Blockchain becomes as natural as URLs

2027: Smart Contract Automation

- Blinks that chain multiple transactions (swap → stake → delegate)
- Complex logic (conditional actions based on wallet data)
- Composable actions (Blinks that invoke other Blinks)
- **Impact**: Sophisticated DeFi strategies expressible as single Blink

2027: Natural Language Blink Construction

- "Create a blink that donates 5 SOL to this address"
- LLM generates endpoint implementation
- Auto-deployment to user's domain

- **Impact**: Non-developer access to Blink creation

9.4 Open Questions

Q: Will X.com provide native Blink support or require extensions indefinitely? - Current: Extension + partial native support - Likely: 2026 Q4 native full support (X benefits from transaction fees) - Timeline: 6-12 months

Q: How do Blinks handle regulatory compliance (KYC/AML)? - Current: No compliance (non-custodial, peer-to-peer) - Future: Optional compliance middleware for regulated entities - Approach: Blinks themselves remain compliant-agnostic; optional wrapper layer

Q: Can Blinks work on L2s like Arbitrum or only Solana? - Current: Solana-only (Dialect SDK focused there) - Future: EVM-based Blink standard (cross-chain standard needed) - Timeline: 2027

Q: How do we prevent Blink spam/scams at scale? - Solutions emerging: community reviews, verified registries, insurance pools - Not solved: requires ongoing ecosystem maturation

---

10. Appendices

10.1 Glossary

Blockchain Action: A shareable blockchain interaction encapsulated as a URL. Also called "Blink" in Solana ecosystem.

Blink Client: The application (X.com, Phantom, Backpack) that renders Blink UI and manages wallet interaction. Also called "Blink consumer".

Blink Provider: Your API that exposes Solana action endpoints. Responsible for transaction construction.

Dialect: The Solana Foundation's standards body for blockchain actions. Maintains `@solana/actions` SDK.

Non-Custodial: User retains private key control throughout. Provider never touches keys.

Solana Actions: The open standard (RFC-style) that defines Blink endpoint formats, CORS handling, and transaction encoding.

Versioned Transaction: Modern Solana transaction format supporting multiple signature types and future extensibility.

Lamport: 1 SOL = 1 billion lamports (smallest unit on Solana, named after Leslie Lamport).

Blockhash: Solana's mechanism to prevent transaction replay. Expires after ~2 minutes.

Devnet: Solana's development network (free SOL, instant confirmation, resets weekly).

Phantom Wallet: Most popular Solana wallet browser extension and mobile app.

Backpack: Alternative Solana wallet with strong Xverse integration.

10.2 Troubleshooting Guide

Problem	Cause	Solution
Extension doesn't detect Blink URL	URL doesn't return JSON or missing CORS headers	Verify endpoint returns `Content-Type: application/json` and `Access-Control-Allow-Origin: *`
Transaction fails to sign in wallet	Blockhash expired (>2 min old)	Call `getLatestBlockhash()` immediately before returning transaction
"CORS error" in browser	Missing OPTIONS endpoint or incorrect headers	Implement OPTIONS handler that returns `Access-Control-Allow-*` headers
Button labels not showing	Metadata missing `links.actions` array	Verify GET response includes complete action definition
Custom amount input doesn't work	Query param not passed to POST	Ensure button `href` includes `?amount={amount}` or parameter name
Wallet shows wrong destination address	Hardcoded address incorrect	Double-check destination wallet in POST handler (typo = lost funds)
Transaction "Insufficient funds" error	User wallet balance < transfer amount + fees	Inform user of minimum balance requirement

---

10.3 Additional Resources

Official Documentation: - Solana Actions Specification — Official standard - Dialect Labs — Action standard maintainers - Phantom Wallet Docs — Implementation details

This Project: - Repository: blink-anti-gravity - Deployment: https://blink-with-anti.vercel.app - Devnet Testing: https://api.devnet.solana.com

Testing Tools: - Solscan — View devnet transactions - SolanaFM — Alternative explorer - dial.to — When available: integrated Blink testing

---

10.4 FAQ

Q: Do I need approval from X.com to create a Blink? A: No. Blinks are open protocol. X.com automatically recognizes URLs with proper metadata.

Q: Can I create a Blink that transfers tokens to multiple addresses? A: Yes. Construct multiple `SystemProgram.transfer()` instructions in a single transaction.

Q: What if my RPC endpoint goes down? A: Implement failover: try primary → fallback → user error message. Consider Alchemy or QuickNode paid tier for uptime SLA.

Q: Do users pay transaction fees? A: Yes, user pays Solana network fees (typically 0.00025 SOL per transaction). Receiver gets full transferred amount.

Q: Can I use Blinks to mint NFTs? A: Yes. Include `createMint()` or metadata program instruction alongside transfer. One transaction = payment + mint.

Q: What happens if a transaction fails? A: User sees error message. No funds transferred (atomic). User can adjust constraints (amount, slippage) and try again.

Conclusion

Blinks represent a fundamental shift in blockchain accessibility. By embedding blockchain transactions directly into social media and removing the friction of wallet-switching, address verification, and manual transaction construction, Blinks bring blockchain capabilities to mainstream users.

This white paper documented the architecture, security model, implementation patterns, and real-world applications of Solana Blinks. The reference implementation (Donate SOL) demonstrates these concepts concretely. As the ecosystem matures—with native platform support, multi-chain expansion, and template libraries—Blinks are positioned to become the standard interaction model between social platforms and blockchain applications.

For developers: Use this documentation as a reference for building your own Blinks. The pattern is standardized, the security model is well-defined, and the tooling improves continuously.

For business stakeholders: Blinks unlock new monetization channels, user engagement models, and distribution mechanisms for blockchain applications. Frictionless transactions drive adoption.

Document Version: 1.0
Last Updated: April 20, 2026
Feedback: Open to community contributions and corrections.